The Empirical Council

Adversarial LLM Review with Hallucination Detection in Solo Security Research

A single-day case study of three filings, fifteen refutations, and the manpage that wasn't

Stuart Thomas

Independent Security Research — Whitby, North Yorkshire, United Kingdom

13 May 2026 · ORCID: 0009-0008-4518-0064 · CC BY 4.0

PLAIN SUMMARY

Solo security researchers have a problem: they read source code faster than they can prove their hypotheses wrong. This paper describes one day of using four AI models as adversarial reviewers before filing any bug report. The models caught fifteen bad candidates. Two of the four also fabricated a quote from an OpenBSD manual page that does not exist. The paper explains how to catch that too.

ABSTRACT

Solo vulnerability research has a signal-to-noise problem that gets worse as the day gets longer. Source-code reading produces high-confidence hypotheses faster than a single human can empirically falsify them; the result is a steady drip of bad submissions to maintainer mailing lists and bug-bounty portals, each of which costs a small slice of researcher reputation and maintainer patience. This paper documents one day's worth of disciplined pre-filing review using a panel of four commodity large language models (DeepSeek-R1, Grok-4, GPT-5, Gemini 3) as adversarial reviewers, with an explicit empirical-verification gate inserted between LLM verdict and submission. In the course of the day, three vulnerability reports were filed; fifteen separate candidate findings were refuted before filing; eleven novel methodology rules were banked; and — to the author's considerable amusement — two of four LLMs were caught hallucinating identical fictitious text from an OpenBSD manual page that has not contained that text since at least the 7.7 release.

Keywords: vulnerability research methodology · large language models · hallucination detection · responsible disclosure · pre-filing review · empirical verification · OpenBSD · Apple Security Bounty

1. INTRODUCTION

The structural pathology of solo security research is that the analyst is also the reviewer. A working professional may read a thousand lines of kernel source in an afternoon and emerge with a confident hypothesis about an unfixed memory leak; the same professional is then expected to be the person who notices that the hypothesis is wrong. This is a difficult ask even when one is fresh, well-fed and not also debugging six hung XPC clients.

The bug-bounty and open-source communities have evolved several immune responses to bad submissions. The Apple PSRT routinely closes reports as "expected behavior" without comment. The OpenBSD developers have a long tradition of public, terse, and educational corrections, of which more below. Each of these immune responses is calibrated to a noise floor that exists primarily because individual researchers do not self-falsify aggressively enough.

This paper documents one day's experiment in installing a more aggressive falsifier between the source-read and the send button. The combination, applied consistently across a working day of nine candidate findings, produced an unusually clean ratio of filings to refutations and surfaced an LLM-hallucination class that the author has not previously seen explicitly named.

The intended argument is that the marginal cost of running four LLMs over a draft submission is small compared to the marginal cost of a maintainer's "get used to disappointment."[1]

2. METHOD

2.1 The Empirical Council

The Council consists of four commodity LLMs accessed via their public chat interfaces: DeepSeek-R1 (source-pattern eye), Grok-4 (adversarial framing), GPT-5 (devil's advocate), and Gemini 3 (pillar / framing).[2]

A "Council prompt" is a structured document in four parts: (1) finding summary; (2) technical evidence — source quotes, disassembly snippets, runtime output, all reproduced verbatim from artefacts the researcher can re-pull; (3) four fixed questions: (a) genuine or by-design? (b) impact material? (c) evidence sufficient to file? (d) correct venue?; (4) symmetric burden of proof — each reviewer must defend any "intended behaviour" reading with concrete documentation.

A `GO` outcome means all four cleared the candidate; `CONDITIONAL-GO` means at least one demanded an additional artefact; `NO-GO` means at least one named a fatal flaw.

2.2 The Empirical Gate

Before the draft email is sent, the researcher attempts to empirically verify each substantive claim the LLMs made — including claims they made *about their own evidence*. Any claim attributed to documentation is re-read from the documentation as it exists *in this session*. Any claim about runtime behaviour is reproduced on the actual target.

This step exists because LLMs hallucinate most confidently when quoting documentation they expect to exist rather than documentation they have read. The classic shape is a paraphrase presented as a

verbatim quote of text that has not existed in the cited document for several major versions.

2.3 The Filing Discipline

The submission text is drafted only after the Council has cleared the candidate and the empirical gate has passed. Every string between quotation marks in the email body is then re-pasted from the source as it exists *now*. This rule emerged later in the day, the hard way.

3. THE XPR-PATTERN SLIP

A useful term for the failure mode this methodology is designed to catch is the *XPR-pattern slip*: a source-derived hypothesis with high confidence that collapses on empirical test. The hypothesis is usually structurally correct ("this function does X") but inferentially wrong ("therefore X is exploitable"). The slip is not in the observation but in the inference, and the only reliable corrective is empirical falsification at the inference step.

Table 1 summarises the day's fifteen refuted candidates. Table 2 summarises the three filed findings.

Table 1. *Candidates examined and refuted, 13 May 2026.*

#	Candidate	Caught by
1	OpenBSD `msgctl` IPC_STAT pointer leak	PoC: kernel never writes the fields claimed leaked
2	iWork QuickLook ZIP64 wrap arithmetic (P2)	Phase 3 PoC: parser bailed before reaching arithmetic
3	Apple Model I/O Phase 2 (9 USDZ PoCs)	QuickLook headless test: all 9 refused without crash
4	Apple Model I/O Phase 3 (Apple-fork diff)	Binary `strings`: Apple cherry-picks upstream fixes
5	macOS `wifivelocityd` Rapport bypass	Dynamic harness: Rapport gates on `com.apple.CompanionLink`
6	macOS Font.mdimporter sfnt name parser	Disassembly: bounds checks are correct
7	macOS Font.mdimporter suitcase Pascal string	Disassembly: disclosure bounded to attacker's own file
8	OpenBSD `iked` sibling-validate hunt	Source read: active audit area, no unfixed siblings found
9	Cross-BSD CVE-2025-0373 (`ifid` overflow)	Source compare: OpenBSD inode 32-bit, struct 16 bytes, no overflow
10	macOS `uarpd` Phase 2 v1 (UARPKit)	Wrong binary: thin NSXPC client wrapper, no parser
11	macOS `uarpd` Phase 2 v2 (CoreUARP)	All four daemons gate listener with `valueForEntitlement:`
12	macOS `imagent` IMDBackgroundMessagingAPIListener	Listener uses anonymous endpoint, not a named mach service
13	macOS `imagent` IMDIncomingClientConnectionListener	Class not instantiated by any shipping binary on 26.4.1
14	macOS `imagent` endpoint-handoff bypass	Only one endpoint-handing method; targets a re-gated service
15	macOS `handwritingd` Phase 7 residency	5.5 GB core: zero hits for injected contact marker

Table 2. *Filings sent, 13 May 2026.*

Finding	Venue	Outcome at end of day
`dasd` `%{public}` log annotations expose entitlement-gated app identifiers to unprivileged processes	Apple Security Bounty	Received, awaiting OE assignment
OpenBSD `ktr_psig()` 4-byte padding-hole stack disclosure	`bugs@openbsd.org`	Reply ×2 from de Raadt; merit not yet ruled on
OpenBSD `pledge(2)` `kill(0, sig)` permitted under any promise via `pid==0` exception	`bugs@openbsd.org`	Closed by de Raadt as intended behaviour

4. CASE STUDY: THE MANPAGE THAT WASN'T

4.1 The Observation

The kernel's gate for `kill(2)` under any pledge is `pledge_kill()` in `sys/kern/kern_pledge.c`. The third branch is a universal exception: `kill(0, sig)` and `kill(getpid(), sig)` are permitted regardless of which promise the process holds. The `pledge(2)` manpage lists `kill(2)` only under `proc`.

```
int pledge_kill(struct proc *p, pid_t pid) {
    if ((p->p_p->ps_flags & PS_PLEDGE) == 0)
        return 0;
    if (p->p_pledge & PLEDGE_PROC)
        return 0;
    if (pid == 0 || pid == p->p_p->ps_pid)
        return 0;
    return pledge_fail(p, EPERM, PLEDGE_PROC);
}
```

4.2 The Hallucination

DeepSeek-R1 and Grok-4 both responded with substantially identical wording:

> *"The manpage for stdio in a current release (including 7.7) already lists `kill(2)` as permitted under stdio, with an explicit qualifier that the pid argument must be 0 or the calling process's PID."*

The author SSH'd into the OpenBSD 7.7 VM and ran `man pledge`. The word "kill" does not appear in the `stdio` section at all. The DeepSeek/Grok claim was, as the methodology section euphemistically puts it, *not present in the source as cited*. GPT-5 and Gemini 3 did not make this claim. All four conditions collapsed to GO after empirical falsification.

4.3 What the Hallucination Was Probably Doing

A reasonable hypothesis is that both models drew on pledge mailing-list discussion or third-party explanations in which the `pid==0` exception is described — combined with a confabulated attribution of that description to the manpage. The shape is the *attribution slip*: the substantive content (the exception exists) is correct; the attribution (the manpage documents it) is not. Detection requires reading the cited source *from the source as it currently exists*.

4.4 The Reply

Two replies arrived from Theo de Raadt within fifteen minutes:

> "This is intended behaviour. In fact it is REQUIRED behaviour. You could try changing it and seeing the effect. Did you? Clearly you didn't. > this is live code diverging from the documented contract. Get used to disappointment."[3]

> "> A stdio-pledged process is, per the documented contract, supposed to only act on already-open file descriptors and process-internal state.
> The manual page does not say that. It does however say the following:
> As a result, all the expected functionalities of libc stdio work."[4]

The second reply identifies a misquote in the submission: the phrase "actions that only occur inside the process" was attributed to the manpage but was the author's paraphrase, inherited from the Council prompt. Rule 10 was banked. A third reply in the `ktr_psig` thread corrected "KASLR" to "KARL", OpenBSD's per-boot relink mechanism.[5]

5. THE ELEVEN RULES

1. **PoC before draft.** A source-derived hypothesis is not a finding until a PoC demonstrates an attacker-observable artefact.
2. **Counter-read the writer.** For any "field X is leaked" claim, identify the write path before filing.
3. **Phase 2 disassembly trails can be wrong.** Phase 3 (empirical trigger) must agree with Phase 2.
4. **Apple forks may be hardened beyond public source.** Verify against the binary, not the upstream.
5. **Framework-level gates may exist beyond binary-local checks.** The broker framework may gate at the dispatch layer.
6. **Anonymous listeners are not reachable without endpoint handoff.** No `machServiceName:` means not bindable from uid=501.
7. **`__objc_stubs` trampolines are not direct calls.** Treating them as direct calls produces wrong reachability analysis.
8. **Name the exact binary, not the framework family.** Confusion between UARPKit and CoreUARP cost half a day.
9. **Use OS-native mitigation terminology.** OpenBSD uses KARL; Apple uses kASLR. Mixing them invites correction.
10. **Re-paste all quoted source from the current source, in the submission session.** No inheritance from earlier prompts.
11. **When one Council voice asks for one tiny extra test, run the test.** The marginal cost is 30 minutes; the alternative is a "Did you?" reply.

6. DISCUSSION

6.1 The Council Is Not a Reviewer Replacement

Two of four reviewers hallucinated identically; the remaining two flagged different conditions. The Council's role is *signal enrichment*, not signal authentication. The authentication step is the empirical gate, always performed by the human.

6.2 The Empirical Gate Is the Whole Point

Any LLM-assisted research workflow that does not include an explicit step for verifying *the LLM's own factual claims against current sources* is a workflow that ships hallucinations in proportion to its output volume.

6.3 Throughput

Three filings, fifteen refutations, eleven rules. The refute-to-file ratio of 5:1 compares to a prior baseline of 2:1. The difference is attributable to the Council-plus-empirical-gate combination.

6.4 Reputation Cost of Filing Noise

Maintainer queues have an informal researcher-reputation model. A researcher whose submissions trend toward "intended behaviour" closures will find their next submissions triaged later. The Council and empirical gate are explicitly throughput-protective: a submission that survives the gate is one whose author can read the reply without flinching.

7. LIMITATIONS

One day, one analyst, three operating systems, roughly nine attack surfaces. The four LLMs are commodity products at specific versions on 13 May 2026; their hallucination patterns are not stable across releases. The XPR-pattern slip is the author's own framing, offered as a diagnostic term rather than a discovery.

8. FUTURE WORK

The most directly useful next study would compare refute-to-file ratios across a panel of researchers using vs. not using the methodology. A second direction: whether attributing a paraphrase to documentation is a recurring LLM failure mode near training cut-off. A third: automating the "re-paste all quoted text" rule via a diff between draft submission and current documentation.

9. ACKNOWLEDGEMENTS

The author thanks Theo de Raadt and the OpenBSD community for prompt and educational replies. No animals were harmed. The fictitious contact "ZXZX-FNORDIUM-MARKER-9942" was deleted from the Parallels VM Contacts database at the conclusion of experimentation.

10. REPRODUCIBILITY

Council prompts and empirical-gate artefacts are available on request.

1. Theo de Raadt, `bugs@openbsd.org`, 13 May 2026.
2. The term “Empirical Council” distinguishes the practice from single-LLM code review; this Council requires verdicts falsifiable against ground truth.
3. Theo de Raadt, `bugs@openbsd.org`, 13 May 2026, 14:50 BST.
4. Theo de Raadt, `bugs@openbsd.org`, 13 May 2026, 14:55 BST.
5. Theo de Raadt, `bugs@openbsd.org`, 13 May 2026, 15:17 BST.

REFERENCES

1. Apple Inc. *macOS 26.4.1 (build 25E253)*, target operating system.
2. OpenBSD Project. *OpenBSD 7.7-release (arm64) / -current*; `pledge(2)` manual page as installed; commits 1b900a0 (deraadt), 76d3556 (dgl), 8cfd528 (millert) cited in the day’s analyses.
3. Pixar Animation Studios. *OpenUSD*, github.com/PixarAnimationStudios/OpenUSD, v25.02.
4. Blacktop. *ipsw*, github.com/blacktop/ipsw, v3.1.680.
5. de Raadt, T. `bugs@openbsd.org`, three replies, 13 May 2026.
6. iVerify; NowSecure; SecurityWeek. *NICKNAME zero-click iMessage exploit*, 2025-06.

Stuart Thomas is a security researcher with prior commits accepted by the OpenBSD project (UNVEIL-01 / `kern_unveil.c` dead-code escalation guard, ok beck@; ELF-07 / `exec_elf.c` vaddr_t truncation, ok guenther@) and submissions accepted into the Apple Security Bounty pipeline.

This paper is dedicated to the proposition that the most useful question a researcher can ask themselves at half past three in the afternoon is “did I just hallucinate that?”